

VideoSpeedControl – User guide

Version 0.9.5

Copyright © 2026 Alex Vulcu. Concept, architecture, direction, and curation by Alex Vulcu.
Implementation assisted by generative AI tools. Licensed under the MIT License.

VideoSpeedControl user guide

VideoSpeedControl gives you full control over the speed of a video clip. Split a clip that was recorded in native slow motion – for example a GoPro at 240 fps or a phone's slow-motion recording – into sections and give each section its own frame rate: speed up the beginning, follow with extreme slow motion, switch to real time, and end in slow motion again.

- **Lossless** needs no re-encoding at all: every original frame is kept and only its playing time changes, so the app makes a variable-frame-rate clip in practically no time and without any loss of quality.
- **Smart re-encode** re-encodes only the sections whose frame rate differs from the output frame rate, in the same codec as the source, which keeps the export fast.
- **Re-encode** encodes the whole clip in a codec of your choice, with the same full control over speed and output frame rate.

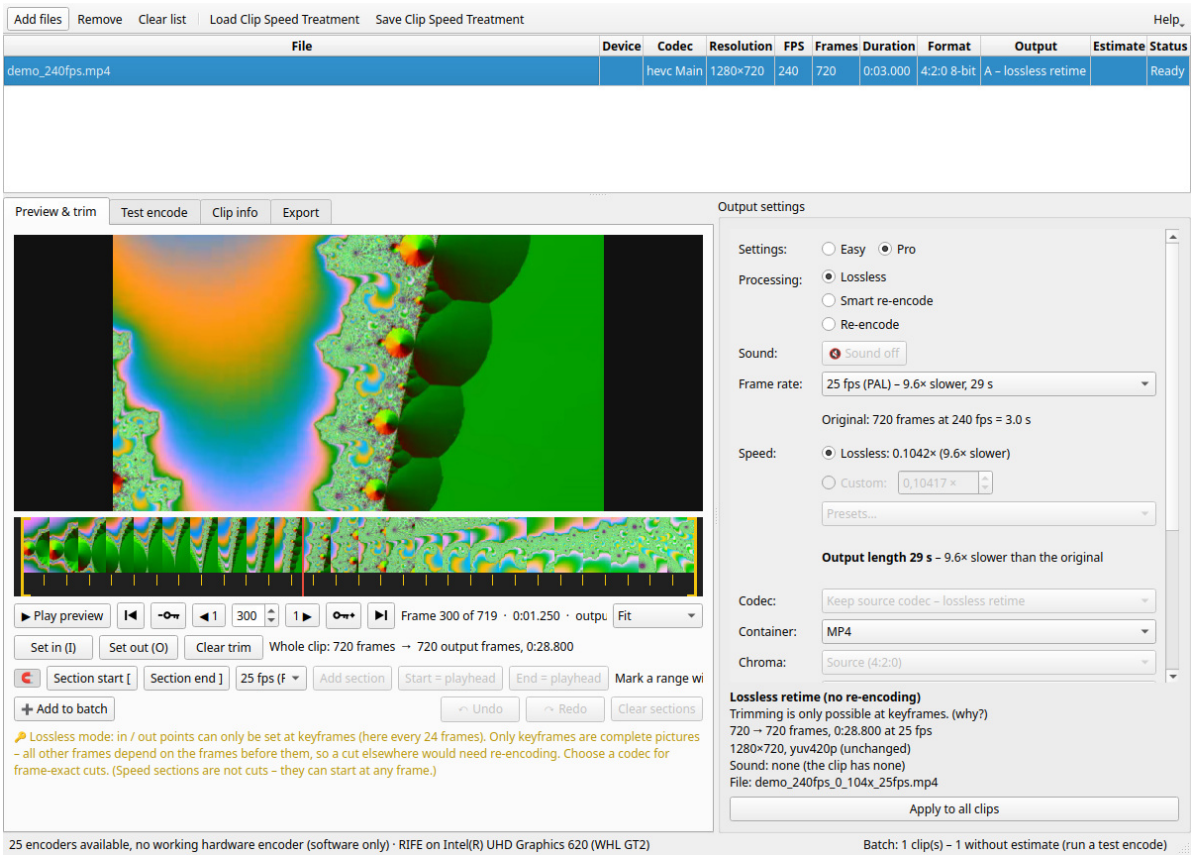
The app saves the speed edits of every clip, so you can resume your work later – or encode the same clip in different codecs to compare them, helped by automatically generated file names with naming options. You can also collect many clips – or several versions of the same clip – in a batch list and export them all in one go.

Contents

1. Quick start
2. The main window
3. Loading clips and clip information
4. Preview and trim
5. Output settings
6. Speed sections
7. Smart re-encode
8. Test encode and time estimates
9. Export and the batch list
10. Convert videos
11. AV2 (experimental)
12. Keyboard shortcuts
13. Tips and troubleshooting
14. Licence and credits

1. Quick start

1. Start VideoSpeedControl and drag one or more video files into the window (or click **Add files...**).
2. Choose the **Processing** mode in the *Output settings* on the right. **Lossless** is the default and the fastest.
3. Pick an output **Frame rate**. The drop-down shows how much slower (or faster) the clip plays and how long it becomes, e.g. *"25 fps (PAL) – 9.6× slower, 1:12 min"*.
4. Press **Space** in the *Preview & trim* tab to watch the result.
5. Open the **Export** tab, choose a folder and click **Start export**.



How slow motion works here

A 240 fps camera records 240 pictures per second. Played at 240 fps, that is real time. Played at 25 fps, the same 240 pictures take 9.6 seconds: the clip is **9.6× slower** – without a single new or missing picture. This is what *Lossless* does. Only when you want a speed that does not fit the frame rate (e.g. 20× slower at 25 fps) do pictures have to be skipped or calculated, which means re-encoding.

2. The main window

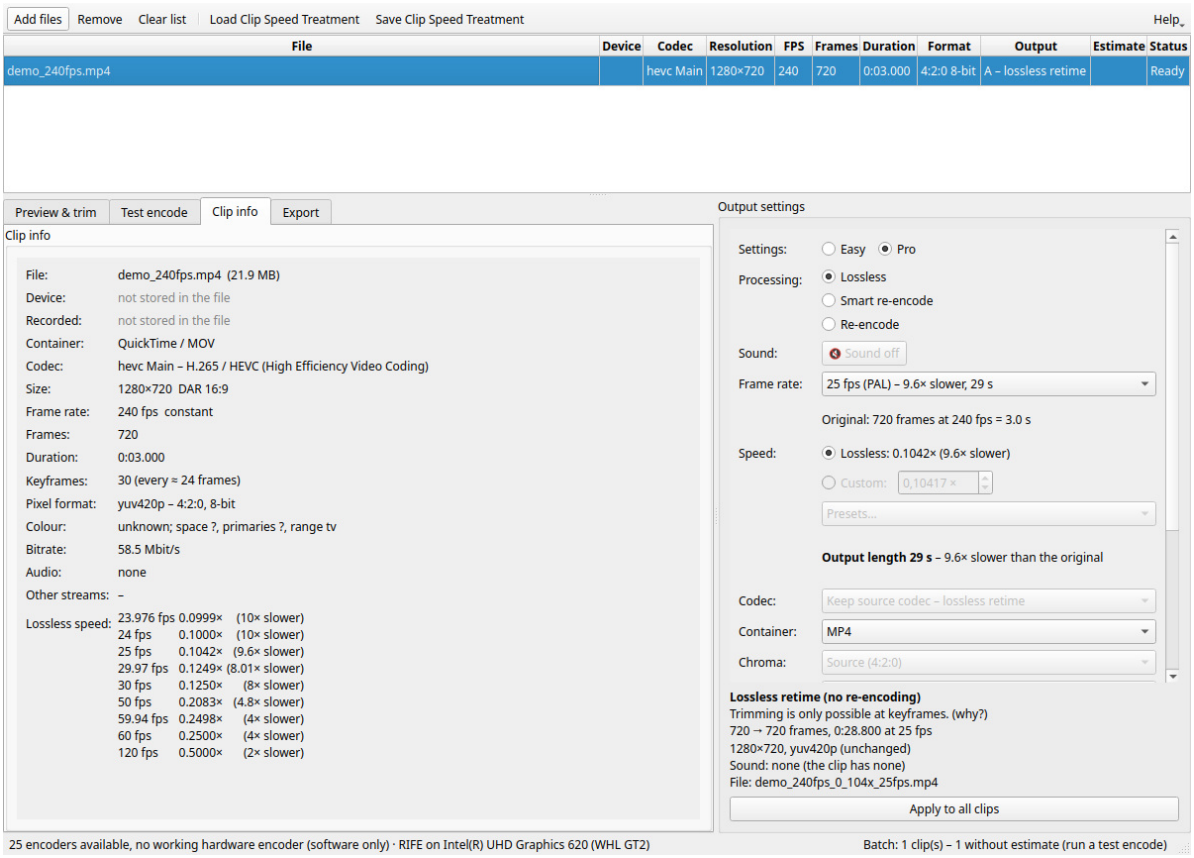
- **Clip list (top):** every loaded file with its device, codec, resolution, frame rate, length, the chosen output and an estimate of export time and size.
- **Tabs (bottom left):** *Preview & trim*, *Test encode*, *Clip info*, *Export*.
- **Output settings (bottom right):** everything about the output of the selected clip, with a summary at the bottom.
- **Help** (right end of the toolbar): this guide (**F1**), the guide as PDF, and *About*.

Each clip has its own settings. **Apply to all clips** copies the output settings of the selected clip to all others (trims and speed sections stay per clip).

3. Loading clips and clip information

Drag files or folders into the window, or use **Add files...**. Any format FFmpeg can read works. After loading, the app scans the clip (exact frame times and keyframes) and builds a small preview copy in the background (*"fast preview 45 %"* next to the frame number).

The **Clip info** tab lists everything about the file: container, codec and profile, resolution and aspect ratio, rotation, frame rate, number of frames, keyframe interval, pixel format, chroma subsampling, bit depth, colour space (SDR / HLG / PQ), bit rate, recording device and date, and audio.



Phone slow motion

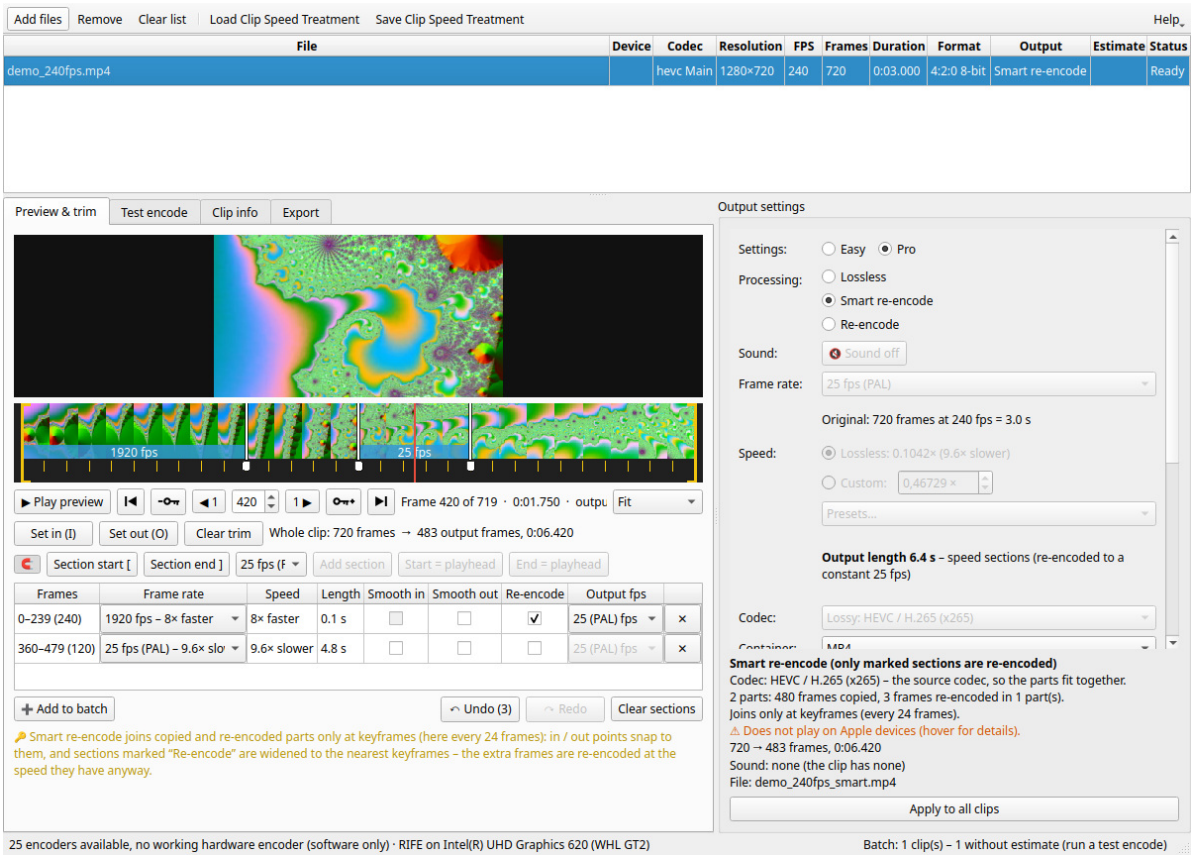
Phones such as the Google Pixel record slow motion at e.g. 240 fps but store it as a 30 fps file that already plays slowly. VideoSpeedControl reads the recording rate from the file and treats **240 fps as the original**: speeds and "× slower" always refer to real time. The clip list shows e.g. *"240 (slow-mo, file 29.97)"*.

Stored previews

The frame list, the timeline thumbnails, the small preview copy used for scrubbing and the keyframe analysis of smart re-encode are stored in the cache folder (Linux *~/.cache/VideoSpeedControl/previews*, Windows *%LOCALAPPDATA%\VideoSpeedControl\previews*). A clip opened again is ready almost at once instead of being scanned and decoded again – also after renaming or moving it, because it is recognised by its content. The preview is stored in small parts; if one is damaged, only that part is made again.

Two minutes after the start – and only while nothing heavy runs – previews nobody needs any more are deleted: a preview stays while its clip is in the list, in the batch list, or has a Clip Speed Treatment (.cst) file. The cache is kept below 2 GB.

4. Preview and trim



Moving around

- Click or drag in the **timeline** (thumbnails with keyframe ticks below them), use the **frame number** box, or the buttons and keys: **◀ 1 / 1 ▶** (←/→) one frame, **– 🔑 / + 🔑** (**Shift+←/→**) previous / next keyframe, **⏮ / ⏭** (**Home/End**) first / last frame, mouse wheel.
- While you drag, the picture comes from the small preview copy – instantly. When the cursor rests for a moment, the sharp full-size frame replaces it.
- **Fit / 100 % / 200 %** zooms the picture; drag to pan, double-click to toggle.

Play preview

▶ Play preview (or **Space**) plays the output at the chosen frame rate and speed – at reduced size and without encoding, so it starts at once. It plays from the cursor to the out-point. Motion interpolation and RIFE are shown as simple blending (they are too slow for real time). If the computer cannot decode fast enough (e.g. 240 fps real time), a note says that the preview runs slower than the export will.

Trim (in / out)

Set in (I) and **Set out (O)** mark the first and last frame of the output; **Clear trim** removes them. The dimmed parts of the timeline are not exported.

In **Lossless** and **Smart re-encode** mode, in and out points **snap to keyframes**. A yellow hint explains why: only keyframes are complete pictures; all other frames store only the differences to their neighbours and cannot be shown once the keyframe before them is cut away. Choose *Re-encode* for frame-exact cuts.

5. Output settings

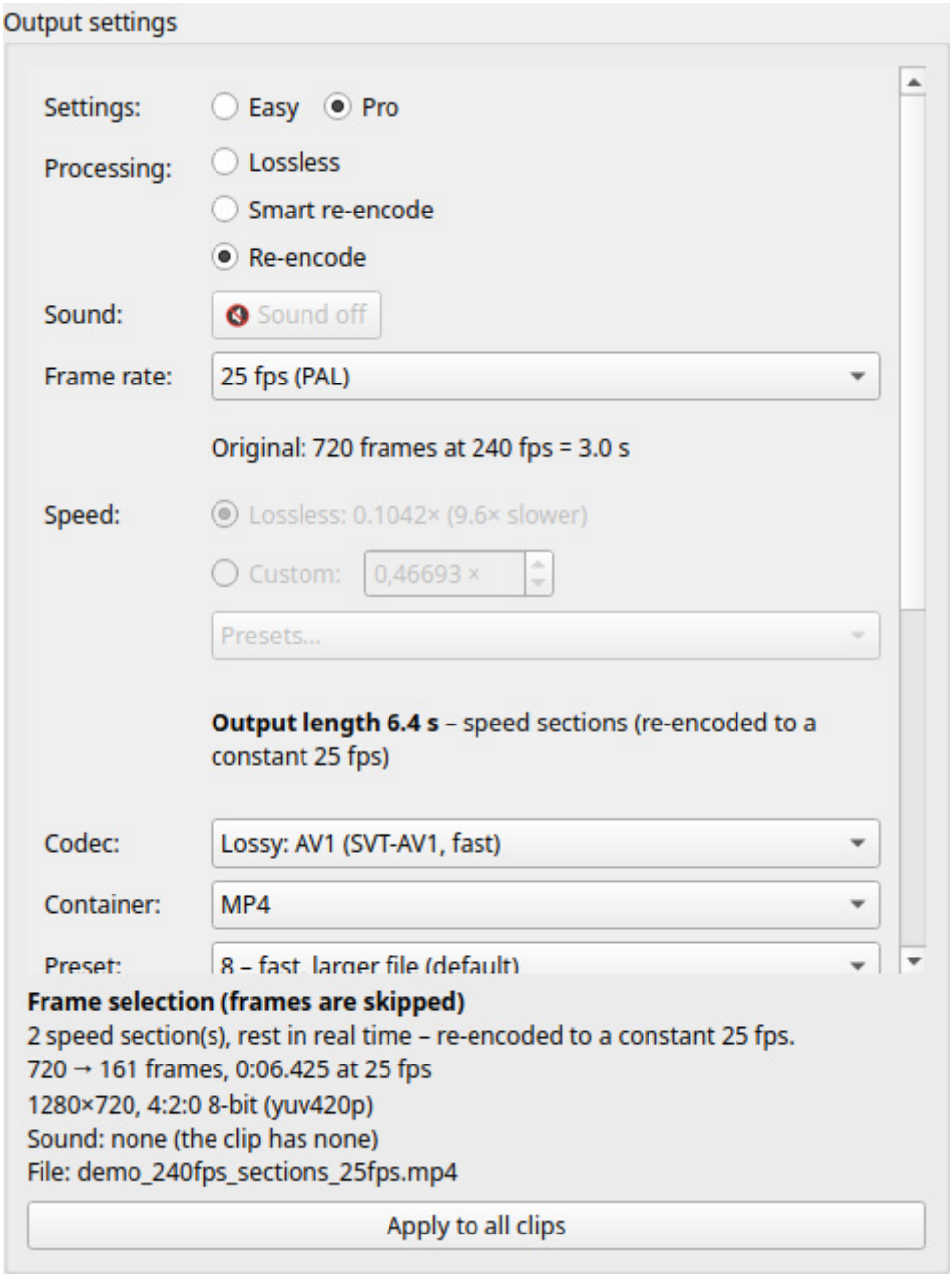
Processing

Mode	What happens	What you can change
Lossless	Every frame stays untouched; only its playing time changes. Seconds instead of minutes, no quality loss.	Frame rate, container, speed sections, trim (at keyframes)
Smart re-encode	Copies the clip and re-encodes only the sections you mark – see chapter 7.	Speed sections with their own output frame rate, quality and preset of the re-encoded parts, container
Re-encode	The whole clip is encoded again.	Everything: codec, size, speed, frame rate, interpolation

Only the settings that the chosen mode can use are enabled.

Easy / Pro (top of the output settings): *Easy* hides the codec details – the codec itself, profile, preset, quality, chroma and bit depth. Smart re-encode then always uses the source codec and re-encode always **AV1 (fast)**, each with its default preset and quality. In Easy every speed change between sections is also **smoothed** automatically (0.5 s, see chapter 6) – the *Smooth in / out* switches appear only in Pro. *Pro* shows every setting (the pictures in this guide show Pro). Switching to Easy applies the easy settings to all clips.

- Choosing **Smart re-encode** marks every speed section whose frame rate differs from its output frame rate for re-encoding; sections that already play at it stay copied (you can change the ticks in the table).
- Switching from Smart to **Re-encode** selects AV1 (fast) again.



Frame rate and speed

- **Frame rate:** the standard rates (23.976 … 120 fps) plus the clip's own rate, marked *(original)*. Each entry shows the slow-down and the output length when every frame is kept.
- **Speed** (Re-encode): *Lossless* uses every frame once. *Custom* sets any speed (e.g. 0.05× = 20× slower) – faster than the lossless speed skips frames, slower calculates in-between frames (see *Interpolation*). *Presets…* offers common values.
- The line below shows the **output length** and how much slower or faster it is than the original.

Codec and container

The codec list is grouped into **Lossless** (FFV1, Ut Video, MagicYUV, HuffYUV, H.264 / HEVC / VP9 / AV1 / AV2 lossless, QuickTime Animation, image sequences), **Lossy** (H.264, HEVC, AV1, VP9, ProRes, DNxHR, CineForm, MJPEG, AV2) and **Hardware** (NVIDIA, Intel, AMD – only offered if a

test encode on your computer worked). Every entry has a tooltip explaining what it is good for. The **container** list only offers formats that can hold the chosen codec. When you switch to *Re-encode*, **AV1 (SVT-AV1, fast)** is preselected – small files at good quality; *AV1 (libaom, best quality)* compresses a little better but is much slower.

Preset

How long the encoder searches for the best compression. At the same quality setting a slower preset gives a **smaller file** and takes **longer**; the visible quality hardly changes. Each entry says what it does, e.g. *"veryslow – slowest, smallest file"*. (For CineForm the row is called *Compression* – there it is a quality level.)

Quality

Named levels that mean about the same for every codec; the codec's own number is shown in brackets:

Level	Meaning
Visually lossless	No visible difference even in a still frame. Large files.
High (default)	Hardly any visible loss – good for further editing and archiving.
Good	Streaming quality (YouTube, Netflix): fine for watching.
Medium	Losses visible on a close look, e.g. in fine detail or dark areas.
Small file	Clearly visible losses – for previews or sending quickly.

Custom… lets you enter the codec's own value. One level down makes the file roughly 30–50 % smaller.

Chroma and bit depth

- **Chroma subsampling:** 4:4:4 (full colour detail – VFX, high-end), 4:2:2 (broadcast TV), 4:2:0 (streaming, Netflix, YouTube). Only combinations the codec supports are offered.
- **Bit depth:** 8-bit or 10-bit. *Like source* keeps what the clip has.

Resolution, aspect and scaling

- **Resolution:** keep the source size, presets from 480p to DCI 4K (including 9:16 and 1:1) or custom.
- **Aspect:** keep the original proportions (fit inside the frame) or force a frame size / aspect ratio; **Fit** decides between black bars (pad), cutting off (crop) or stretching.
- **Scaling:** the resampling method with quality and speed ratings – Lanczos is the sharpest, Area the cleanest for strong downscaling.

Interpolation

Used when the speed is slower than the lossless speed and new in-between pictures are needed:

- **Frame duplication** – repeats pictures; motion stutters. Fastest.
- **Frame blending** – cross-fades neighbours; smooth but with ghosting.
- **Motion interpolation (minterpolate)** – calculates real in-between pictures on the CPU; good for slow, even motion. Very slow.
- **RIFE (AI)** – neural network on the graphics card (Vulkan, also integrated graphics); clearly better motion. Works on 8-bit RGB. Not available together with speed sections or AV2.

Sound

The **Sound** button right under *Processing* switches the sound on or off with one click: 🔊 **Sound on – pitch preserved** (default) or 🔇 **Sound off**. With sound on, the app takes the first audio track of the clip and stretches or compresses it to the new length of the video – the **pitch stays the same**, so there are no deep or squeaky voices. Speed sections get their own stretch factor each, so the sound follows every section. Very strong slow motion makes speech hard to understand (it is simply 10× slower), but it keeps its pitch.

Phone slow motion: phones store the sound of a slow-motion clip slowed down like a tape – it sounds deep. VideoSpeedControl first brings it back to its real pitch and real-time length, then stretches it to the video.

The sound is added after the video is finished, so a **lossless** video stays lossless; only the audio track is encoded (AAC in MP4 / MOV / MKV, Opus in WebM, PCM in AVI / MXF). Image sequences and AV2 files (FFmpeg cannot handle AV2 yet) stay without sound – the button is greyed out then, and its tooltip says why.

6. Speed sections

Speed sections give parts of a clip their own frame rate – for example the jump in slow motion, the run-up in real time, the landing 8× faster.

1. Move the cursor to the first frame and click **Section start [** (key **[**).
2. Move to the last frame and click **Section end]** (key **]**). A dashed frame marks the range.
3. Choose the frame rate (from 23.976 fps up to **8× faster** than the original) and click **Add section**.

Quicker: skip step 1. Without a marked start, **Section end]** starts the section right after the section before it – or at the start of the clip (trim) if there is none – so sections follow each other without gaps. Inside an existing section it starts at that section's start (the section is split).

Sections appear as blue bands in the timeline and in the table below it, where you can change their frame rate, jump to them (click) or remove them (✕). Everything outside sections plays in **real time**. A new section that overlaps an existing one wins.

- **Editing a section:** click it in the timeline or in the table to select it. **Start = playhead** / **End = playhead** move its first / last frame to the cursor; the frame rate is changed in the table.
- **Moving a border:** the borders between sections (and between a section and real time) are white lines with a grip below the thumbnails. Drag a border to make one section longer and its neighbour shorter; the picture shows the frame at the border while you drag. One drag is one undo step.
- **Smooth transitions:** tick **Smooth in** / **Smooth out** in the table to let the speed change evenly instead of jumping – e.g. from 9.6× slow motion to real time. The change takes **0.5 s** of the output and is centred on the border (shorter if a section is too short); the timeline shows it as a dark field with a slanted line (rising = getting faster). It works in every processing mode; in smart re-encode the transition is re-encoded when a section next to it is.
- **Magnetic cursor 🧲:** when you drag near a section, the cursor snaps to the frame right after it (to continue seamlessly) or – while you mark an end – to the frame right before the next section. Click 🧲 to switch it off.
- **Clear sections** (right) removes all sections after asking.
- ↶ **Undo** (**Ctrl+Z**) and ↷ **Redo** (**Ctrl+Shift+Z** or **Ctrl+Y**) step back and forward through every change to the sections – adding, changing, moving, removing, clearing, loading – up

to 200 steps, separately for each clip. The number on the button shows how many steps you can go back (or forward); hover over it to see which change it undoes.

Saving and loading a Clip Speed Treatment (.cst)

Save Clip Speed Treatment… in the toolbar (**Ctrl+S**) saves the current clip's sections, trim and output settings as a *Clip Speed Treatment* file (`.cst`). It is offered in *Documents/VideoSpeedControl/Clip Speed Treatments* by default – the Android app saves there too. **Load Clip Speed Treatment…** (**Ctrl+L**) – or dropping a `.cst` onto the window – loads it again. The Android app reads and writes the same files.

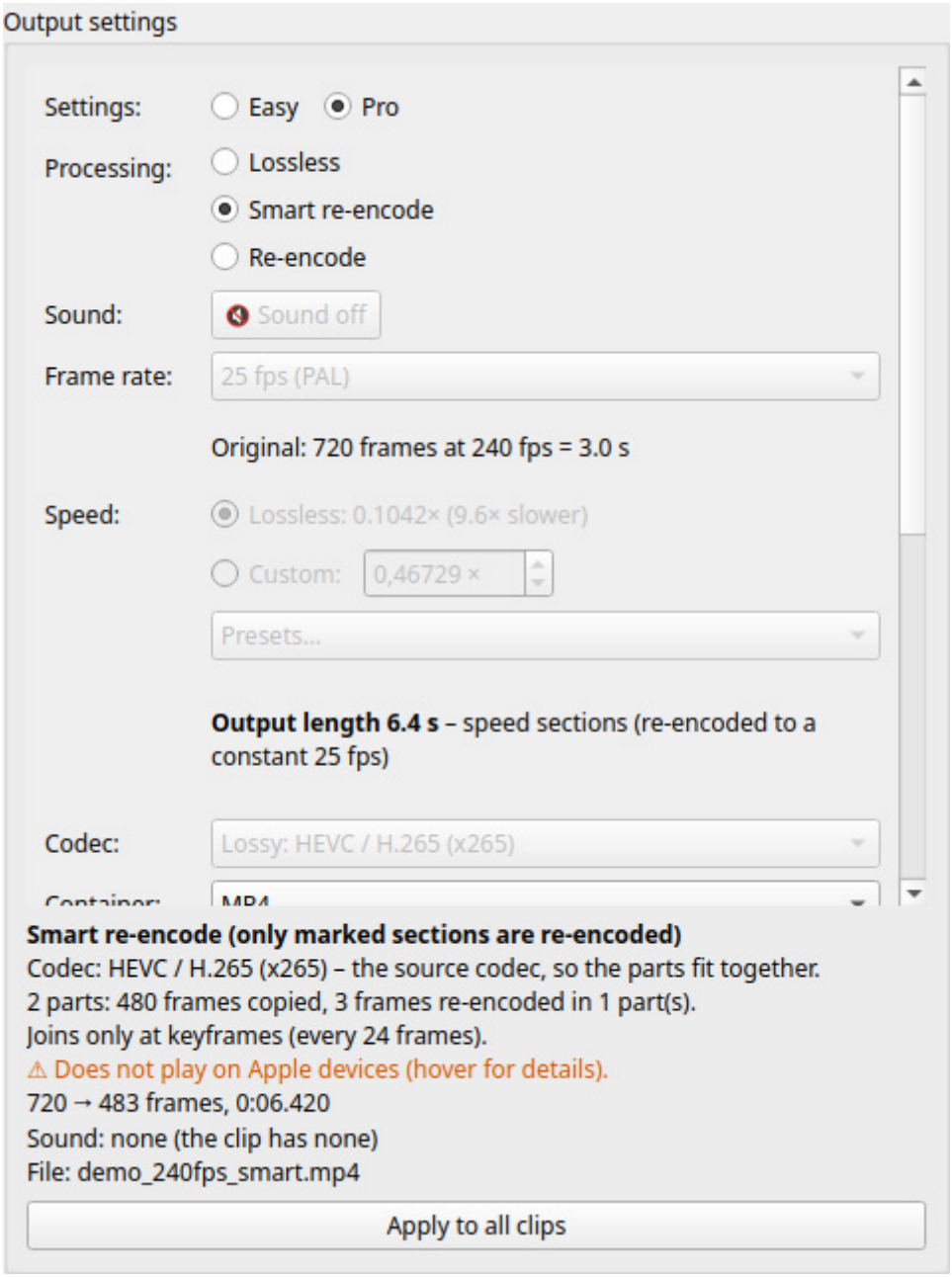
A treatment belongs to exactly one video: it stores a fingerprint of the file's content and only loads onto that video. The file name does not matter – a renamed or moved copy still matches. If the video is not in the list yet but is still where it was when you saved, it is added automatically. A treatment never loads onto a different video, because its frame numbers would not fit.

How sections are exported depends on the processing mode:

- **Lossless:** every frame is kept with its own playing time – one file with a **variable frame rate**. Sections can start at any frame. Plays in all common players; editing programs prefer a constant frame rate. Very fast sections (e.g. 1920 fps) make players decode that many frames per second – choose re-encoding for smooth playback there.
- **Re-encode:** the whole clip is converted to the chosen **constant frame rate**; fast sections skip pictures, slow ones get in-between pictures.
- **Smart re-encode:** see the next chapter.

7. Smart re-encode

Smart re-encode combines both: sections marked **Re-encode** in the section table are encoded with the **source codec** at their own **Output fps**; everything else is copied unchanged. All parts are joined into **one file**.

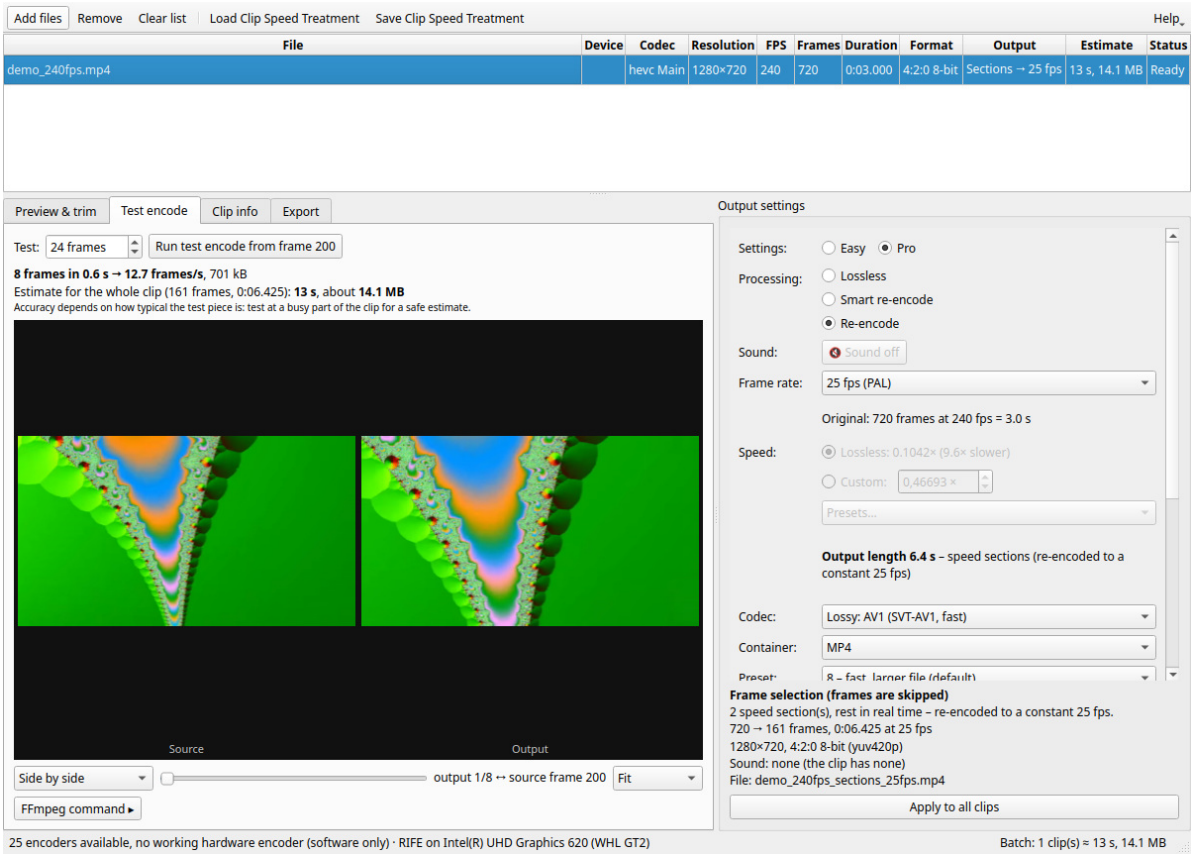


Things to know:

- **Only for H.264 and HEVC footage** (e.g. GoPro, phones). The codec is fixed to the source codec – otherwise the parts could not be joined.
- **Joins only at keyframes.** In / out points snap to them, and re-encoded sections are widened to the nearest keyframes automatically – the extra frames are re-encoded at the speed they have anyway. Some cameras write "open" keyframes where the following frames still depend on earlier ones; the app detects them when loading the clip and does not join there.
- **Not playable on Apple devices.** The joined file changes the decoder settings inside the stream (MP4 tag *hev1* / *avc3*). QuickTime, iPhone, iPad and Final Cut cannot play this – VLC, mpv, Windows, Android, Chrome and Firefox can. For Apple, re-encode the whole clip instead.
- The output frame rate you chose last becomes the default for the next re-encoded section.

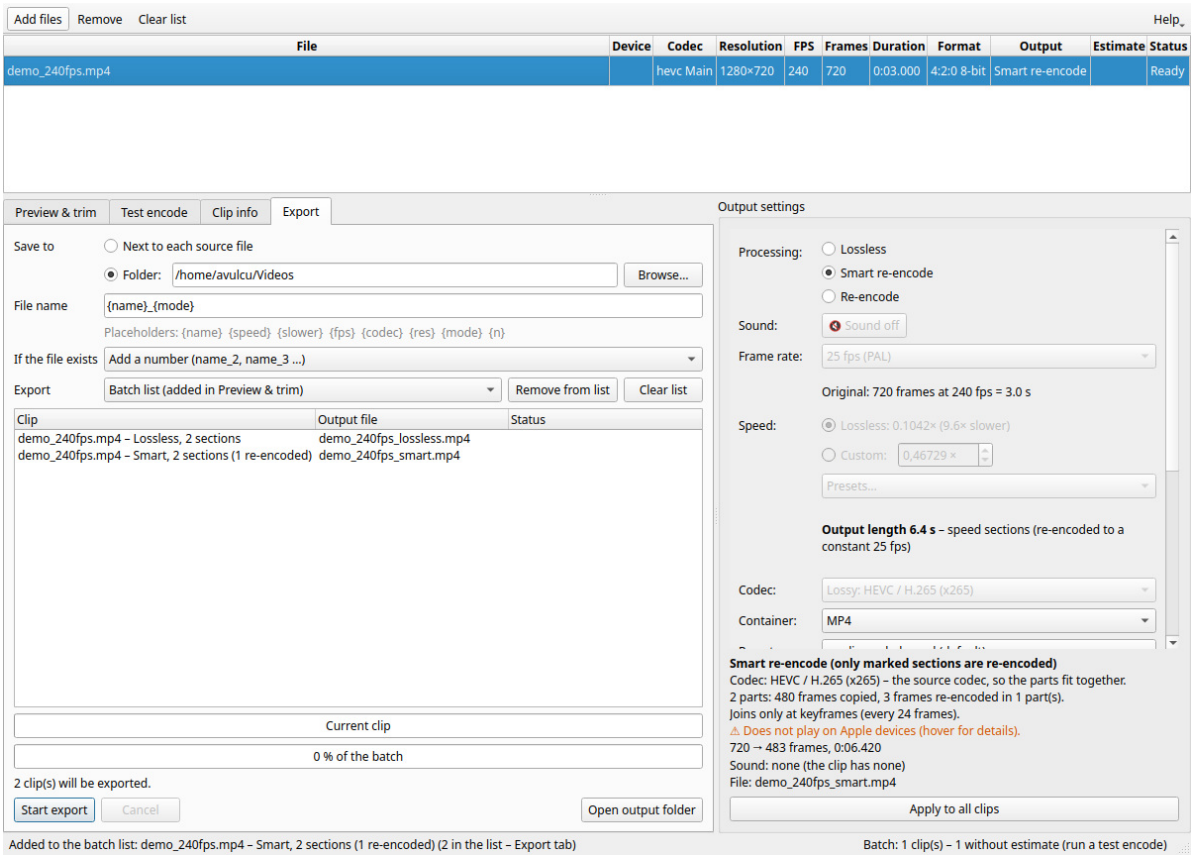
8. Test encode and time estimates

The **Test encode** tab encodes a few frames (default 100) from the cursor position with the exact output settings. It shows the encoding speed, the file size and an **estimate for the whole clip**, and compares source and output **side by side** or with a **wipe** – zoomable to 100 % and 200 %. Test at a busy part of the clip for a safe estimate. The *FFmpeg command* button shows what is run.



Estimates also appear in the clip list. Clips with the same settings share a test, scaled by their size.

9. Export and the batch list



- **Save to:** next to each source file, or a folder.
- **File name:** a pattern with placeholders – `{name}` source name, `{speed}` speed factor, `{slower}` how many times slower, `{fps}` output frame rate, `{codec}`, `{res}` output size, `{mode}` lossless / smart / reencode, `{n}` position in the queue.
- **If the file exists:** add a number, overwrite, or skip.
- **Export:** all clips, the selected clips, or the **batch list**.
- Progress is shown per clip and for the whole batch; **Cancel** stops it, and incomplete files are deleted.

Batch list

➕ **Add to batch** (Preview & trim tab) saves the clip exactly as it is at that moment – processing mode, settings, trim and all sections – as an entry in the batch list. The sections stay in place, so you can add the same clip several times, e.g. once *Lossless* and once *Smart re-encode*. Later changes to the clip do not change entries already in the list. The list stays after the export; **Remove from list** and **Clear list** manage it. Entries that were exported successfully are marked **Done** and are not exported again when you click **Start export** later – only entries added (or failed / cancelled) since then run. To export a done entry again, change it with *Edit this entry*, or delete its output file. Use `{mode}` in the file name to tell the versions apart.

Double-click an entry in the batch list to open it again: its sections, trim and settings are loaded into Preview & trim (the clip's previous sections can be brought back with ↶ Undo). You are asked what you want to do:

- **Another encoding** – change codec, preset, quality or size and click ➕ **Add to batch**: a new entry is added, the old one stays.
- **Edit this entry** – change it and click ✔ **Update batch entry**: the entry is replaced.

Whenever you add a clip that is already in the batch list, VideoSpeedControl asks whether you want another encoding and offers **name additions** with checkboxes – codec, preset, quality and resolution, e.g. `jump_x265_slow_high_1920x1080.mp4`. A preview shows the resulting file name; your choice of boxes is remembered.

10. Convert videos

Convert videos… (toolbar) re-encodes many videos at once – for example a folder of old AVI / DivX files into a modern codec. It is independent of the clip list and the speed settings: every frame is kept once, the length stays the same, and the originals are never changed. Add files or whole folders (with sub-folders if you like) or drop them onto the window. The new files go into a *Converted* folder next to each original, or into a folder you choose; existing files are never overwritten.

Easy (top right): codec, quality level and *Keep the sound* – plus **Smart**.

Pro shows every setting:

Setting	What it does
Frame rate	*Original* (each video keeps its own) or a rate from the list. Another rate keeps the length: frames are dropped or repeated, never invented.
Container	MP4, MOV or MKV. AV1 cannot be stored in MOV, so MOV is greyed out for AV1.
Codec	AV1 (SVT-AV1 or libaom), HEVC, H.264.
Picture	*Quality level*, an *own quality value* (the codec's CRF), a *data rate* in kbit/s, or *Smart*.
Sound	*Smart*, *Copy unchanged*, AAC, Opus, MP3, FLAC or *No sound*, with **channels** (like the original, mono, stereo, 5.1, 7.1 – never more than the original has) and the **sound data rate**.

The settings are the **general settings** for every video. Select one or more videos and click **Own settings for selected** to give them their own settings – the Settings column then shows *Own: …*; select them again later to change them, or click **Back to general settings**. Combinations that do not work are marked with ⚠ in the list (for example copying MP3 sound into MOV), and converting waits until they are fixed.

Smart: smaller files with the same picture and sound

With **Smart** every video is measured before it is converted: three short samples are encoded at several quality values and compared with the original by **VMAF** (a perceptual score from 0 to 100). The smallest file that still reaches the goal is used:

- **Looks identical** (default): no visible difference (VMAF ≥ 96 on average, every sample ≥ 94.5).
- **Smaller, minimal loss**: noticeably smaller, a slight loss only on a close look (VMAF ≥ 93).

The sound is **copied unchanged** when the container can hold it – nothing is lost and it is the smallest choice; otherwise it becomes AAC with the same channels and sample rate at a data rate matched to the source (for example MP3 96 kbit/s → AAC 80 kbit/s). The Settings column shows the result before you convert, e.g. *quality 27 ≈ 145 kbit/s, ~28 %*. **Measure now** measures without converting; *Convert* measures first anyway. Measuring takes about 20 seconds for a small old video and longer for HD.

A re-encode can never be better than its source – Smart keeps what is there and saves the space. Example: old DivX cartoons (368×208, 10 fps) became AV1 files of about 30–40 % of their size with the MP3 sound copied.

11. AV2 (experimental)

AV2 is the successor of AV1 (AOMedia, specification 1.0 from May 2026), about 30 % smaller at the same quality. FFmpeg cannot handle AV2 yet, so the app uses the AVM reference software. It is a research tool and **extremely slow** – about 75 seconds per 1080p frame on a laptop CPU, about 20 seconds at 360p – so use it for short clips and small sizes. Test encodes use at most 5 frames. Output is IVF or WebM; hardly any player can play AV2 yet. On Linux the encoder is built with `tools/fetch_avm.py`; without it, AV2 is shown as unavailable.

12. Keyboard shortcuts

Key	Action
F1	This user guide
Ctrl+O	Add files
Delete	Remove the selected clips
Space	Play / stop the preview
← / →	Previous / next frame
Shift+← / →	Previous / next keyframe
Home / End	First / last frame
I / O	Set in / out point
[/]	Section start / end
Ctrl+Z	Undo the last change to the sections
Ctrl+Shift+Z / Ctrl+Y	Redo
Ctrl+S	Save Clip Speed Treatment (.cst)
Ctrl+L	Load Clip Speed Treatment (.cst)

13. Tips and troubleshooting

- **Which mode?** Lossless whenever the frame rates fit – it is instant and perfect. Smart re-encode when only some parts need a constant frame rate. Re-encode for a different size, codec, or editing-friendly constant frame rate.
- **Editing programs and variable frame rate:** DaVinci Resolve and Premiere handle variable-frame-rate files poorly. Export with Re-encode (e.g. ProRes or DNxHR) for editing.
- **The preview stutters:** very high frame rates (240 fps real time) need more decoding power than many laptops have – the export is not affected.
- **No hardware encoder:** hardware encoders are only offered when a test encode on your computer works; software encoders always work.
- **RIFE is slow or unavailable:** RIFE needs a Vulkan-capable graphics card; on integrated graphics it is slow, without Vulkan it is not offered.
- **The sound is hard to understand:** that is normal for strong slow motion – 10× slower speech keeps its pitch but is stretched. Switch the sound off (🔇) or add music later.
- **Smart re-encode is greyed out:** the clip is not H.264 / HEVC, the keyframe analysis is still running, or no keyframe allows a join.

14. Licence and credits

Copyright © 2026 Alex Vulcu. Concept, architecture, direction, and curation by Alex Vulcu. Implementation assisted by generative AI tools. Licensed under the MIT License – see the LICENSE file.

VideoSpeedControl uses other open-source software with its own licence: **FFmpeg** (GPL build, includes x264 / x265), **Sonic** for stretching the sound (Apache-2.0, the same algorithm as the Android app), **Qt / PySide6** (LGPL), **rife-ncnn-vulkan** with the RIFE model (MIT) and **AVM**, the AV2 reference software (BSD-3-Clause-Clear). Details are in THIRD_PARTY_NOTICES.md.